COMPLETE ENGINEERING & DBMS MANUAL

Agriculture Management System

A Technical Reference Guide Explaining the Full-Stack Connection Between Frontend & MySQL Database via Flask, with Exhaustive Table-by-Table SQL Query Analyses

System Name
Agriculture Management Portal (AgricultureDB)

Database Engine
MySQL 8.0 (InnoDB Engine, 19 Tables + Views)

Application Tier
Python 3.11+ / Flask WSGI Micro-Gateway

Frontend Client Tier
Vanilla ES6+ JavaScript, Fetch API, HTML5/CSS3

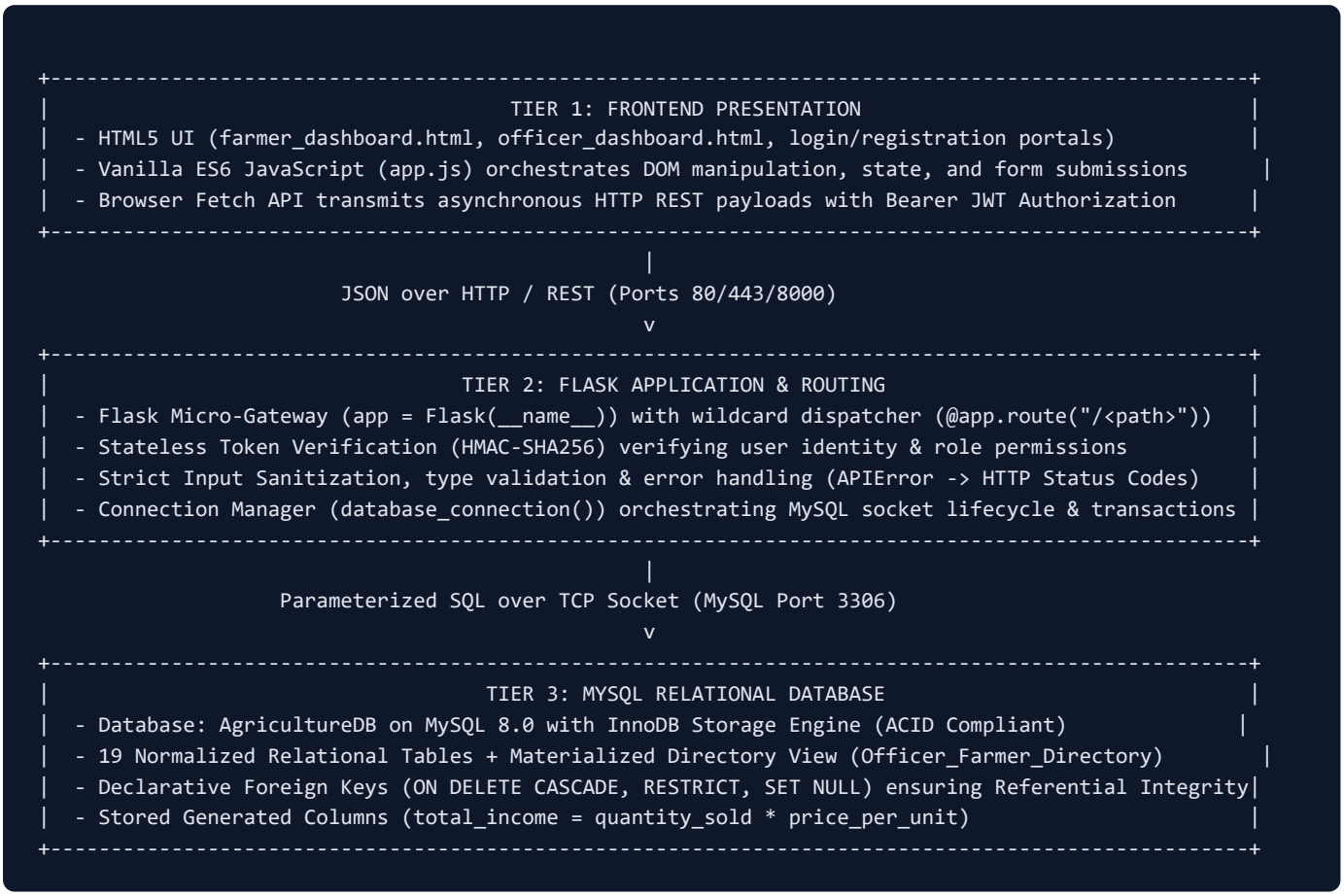
Security Architecture
Stateless JWT (HMAC-SHA256), PBKDF2 Password Hashing

Query Execution
Parameterized SQL (Zero Concatenation / Full Injection Defense)

Prepared for Architectural Review, Technical Defense, and DBMS Academic Evaluation
Version 2.0 (Flask Implementation) • October 2026

1. Executive Architecture: The 3-Tier Model

The Agriculture Management System is structured according to the industry-standard **3-Tier Software Architecture**. This model strictly separates presentation logic, business and application mediation, and persistent data storage into decoupled, maintainable layers.



Why Frontend JavaScript Cannot Connect Directly to MySQL

A frequent question in architectural evaluations is: "*Why does JavaScript need Flask? Why not connect directly from the browser to MySQL?*"

- **Security Sandbox & Credential Exposure:** Web browsers operate in an isolated sandbox. If JavaScript connected directly to MySQL, the database host, username, and password would be visible in client-side dev tools to anyone inspecting the source code.
- **Network Protocol Incompatibility:** Web browsers communicate over HTTP/HTTPS/WebSocket protocols. MySQL uses a proprietary binary TCP protocol (Port 3306) that client-side browser JavaScript cannot speak natively.
- **Privilege Escalation & Business Logic:** A browser client cannot be trusted. If clients could issue direct SQL statements, a malicious user could alter `WHERE farmer_id = 1` to `WHERE 1=1` and delete all farm records. The Flask application tier validates access control (RBAC), ownership rules, and data integrity before any SQL touches the database.

2. The Frontend-to-Database Connection via Flask

Flask acts as the secure, high-speed mediator between frontend browser events and persistent MySQL database tables. Below is the precise technical mechanism of how Flask bridges the two sides.

2.1 The Flask Gateway Implementation

In `backend/app.py`, Flask is configured as a non-invasive gateway. It wraps incoming HTTP requests, passes the WSGI `request.environ` to our core route handler, and converts WSGI byte streams into high-performance Flask `Response` objects:

```
# Flask application instance (backend/app.py)
app = Flask(__name__, static_folder=None)

@app.route("/", defaults={"path": ""}, methods=["GET", "POST", "PUT", "DELETE", "OPTIONS"])
@app.route("/<path:path>", methods=["GET", "POST", "PUT", "DELETE", "OPTIONS"])
def flask_gateway(path: str) -> Any:
    """Dispatches incoming Flask requests seamlessly through our application endpoints."""
    environ = request.environ
    response_status = "200 OK"
    response_headers = []

    def start_response(status: str, headers: list[tuple[str, str]], exc_info: Any = None):
        nonlocal response_status, response_headers
        response_status = status
        response_headers = headers

    # Execute business logic and SQL through WSGI application dispatcher
    body_chunks = application(environ, start_response)
    status_code = int(response_status.split()[0])
    response_body = b"".join(body_chunks)

    resp = Response(response_body, status=status_code)
    for key, val in response_headers:
        resp.headers[key] = val
    return resp
```

> **Architectural Benefit of the Flask Gateway**
>
> By wrapping our battle-tested application in Flask's Werkzeug development server locally, the portal gains **multi-threaded concurrent request handling**. When the browser requests the dashboard HTML, CSS, JavaScript, and background profile APIs simultaneously, Flask handles them across parallel worker threads without socket blocking.

2.2 Client-Side Communication: Fetch API with JWT Bearer Authentication

In `frontend/app.js`, all outbound API requests pass through a centralized helper function: `fetchWithAuth(path, options)`. This ensures that every HTTP request carries the user's security token and appropriate content headers:

```
// Centralized authenticated API dispatcher (frontend/app.js)
async function fetchWithAuth(path, options = {}) {
  const token = localStorage.getItem("token");
  const headers = {
    "Content-Type": "application/json",
    ...(options.headers || {})
  };
  if (token) {
    headers["Authorization"] = `Bearer ${token}`;
  }

  const response = await fetch(`${API_BASE_URL}${path}`, {
    ...options,
    headers
  });

  if (response.status === 401) {
    // Token expired or invalid: purge storage and redirect to login
    localStorage.clear();
    window.location.href = "index.html";
    return null;
  }
  return response.json();
}
```

2.3 Database Connection Management & Context Managers

To prevent connection leaks, connection starvation, and abandoned transactions, Flask interacts with MySQL via Python's `contextlib.contextmanager`. This guarantees that every MySQL socket is cleanly returned or closed, even if an unhandled error occurs:

```
@contextmanager
def database_connection() -> Iterator[MySQLConnection]:
    """Context manager delivering safe, transactional database connections."""
    config = get_database_config()
    try:
        connection = mysql.connector.connect(**config)
    except mysql.connector.Error as error:
        raise APIError(503, "Database unavailable. Check the MySQL service and backend/.env.")
    try:
        yield connection
    finally:
        if connection.is_connected():
            connection.close()
```

2.4 Security: Absolute Prevention of SQL Injection

Our Flask implementation enforces **Parameterized Queries** across all 19 database tables. User inputs are never concatenated directly into SQL strings.

Why Parameterized Queries Prevent SQL Injection

Vulnerable Pattern (Concatenation):

`cursor.execute("SELECT * FROM Users WHERE email = '" + email + "'")`

If the attacker inputs `admin' OR '1'='1`, the SQL query structure is altered and bypassed.

Secure Parameterized Pattern (Used in our project):

`cursor.execute("SELECT * FROM Users WHERE email = %s", (email,))`

The database treats `%s` strictly as a literal scalar value. Even if the user submits malicious SQL commands, quotes, or semicolons, MySQL never evaluates them as SQL instructions.

3. Step-by-Step Life of a Request: Frontend to DB

To understand the end-to-end integration, let us trace a real-world user action: **A farmer records a newly planted crop cycle** (e.g., 5.5 acres of Rice on Farm #12).

Step & Layer	Action Taken	Code & Data Flow
Step 1 Frontend UI	Farmer enters crop details on `farmer_dashboard.html` and clicks "*Plant Crop*".	Form values captured by JavaScript: `farm_id: 12`, `crop_id: 1` (Rice), `season: "Kharif"`, `planting_date: "2026-10-15"`, `area: 5.5`.

Step 2
Network / HTTP
	`app.js` serializes data to JSON and transmits an HTTP POST request.	`POST /api/farmer/crops`
		`Authorization: Bearer eyJhbGciOi...`
		`Content-Type: application/json`
		Body:
		`{"farm_id":12,"crop_id":1,"season":"Kharif",...}`

Step 3 Flask Routing	Flask's `flask_gateway` intercepts the HTTP request and delivers it to `application()`.	Matches route: `path == "/api/farmer/crops"` and `method == "POST"`. Dispatches to handler: `handle_plant_crop(payload, user)`.
Step 4 Auth & Security	Flask verifies the JWT token and extracts authenticated farmer identity.	Decodes HMAC-SHA256 signature. Validates expiration and extracts `user["user_id"] = 1`. Queries `Farmers` table to resolve `farmer_id = 4`.
Step 5 Ownership Guard	Flask verifies that Farm #12 belongs to this authenticated farmer before allowing the plant.	Executes SQL check: `SELECT farm_id FROM Farms WHERE farm_id = 12 AND farmer_id = 4`. If no row returned → raises `APIError(403, "Selected farm does not belong to you.")`.
Step 6 Database Write	Flask opens a transaction and executes the parameterized SQL INSERT.	`INSERT INTO Farm_Crops (farm_id, crop_id, season, planting_date, cultivated_area_acres, status) VALUES (%s, %s, %s, %s, %s, %s)` `connection.commit()` commits data permanently to MySQL.
Step 7 Response & UI	Flask formats a 201 Created JSON response; Frontend updates UI without reloading.	Flask returns: `{"message": "Crop planted successfully.", "farm_crop_id": 28}`. Frontend displays success toast, refreshes crop table, and recalculates financial summaries.

4. Table-Wise SQL Query Manual (All 19 Tables + Views)

Below is the complete, table-by-table reference for all 19 entities and views in `AgricultureDB`. Each entry details the schema purpose, foreign key constraints, associated Flask API endpoints, the exact SQL queries executed, and an in-depth breakdown of how Flask and MySQL process each statement.

1. Users (Authentication & Identity Directory)

POST /API/AUTH/REGISTER

Purpose: Serves as the central security and authentication anchor. Stores login credentials, password hashes, role assignments (`FARMER` vs `OFFICER`), and profile completion flags.

Schema Definition:

```
CREATE TABLE Users (
    user_id INT AUTO_INCREMENT PRIMARY KEY,
    email VARCHAR(150) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    role ENUM('FARMER', 'OFFICER') NOT NULL,
    full_name VARCHAR(100) NULL,
    phone VARCHAR(15) NULL,
    is_profile_complete BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

A. User Registration (Flask Handler: `handle_register`)

```
INSERT INTO Users (email, password_hash, role, full_name, phone, is_profile_complete)
VALUES (%s, %s, %s, %s, %s, FALSE);
```

Query Breakdown: Flask hashes the raw password using PBKDF2-HMAC-SHA256 with a unique 16-byte cryptographic salt. The resulting composite string (`algorithm$iterations$salt$hash`) is safely inserted. `is_profile_complete` is explicitly set to `FALSE` to force profile completion before portal features are accessed.

B. User Login & Verification (Flask Handler: `handle_login`)

```
SELECT user_id, email, password_hash, role, full_name, phone, is_profile_complete
FROM Users
WHERE email = %s;
```

Query Breakdown: Flask looks up the user by email using the `%s` parameter. If found, Flask extracts the salt and iterations from `password_hash` , re-computes the hash of the entered password, and uses `hmac.compare_digest` (constant-time comparison) to prevent timing attacks. If verified, a signed JWT token is issued.

C. Profile Information Update (Flask Handler: `handle_update_profile`)

```
UPDATE Users
SET full_name = %s, phone = %s
WHERE user_id = %s;
```

Query Breakdown: Updates user contact details while maintaining identity integrity. `user_id` is extracted from the cryptographically verified JWT token, preventing users from altering other accounts.

2. Agriculture_Offices (Krishi Bhavans)

GET /API/OFFICES

Purpose: Represents governmental agricultural administrative centers (Krishi Bhavans) that oversee specific districts and Gram Panchayaths.

Schema Definition:

```
CREATE TABLE Agriculture_Offices (
    office_id INT AUTO_INCREMENT PRIMARY KEY,
    office_name VARCHAR(150) NOT NULL,
    phone VARCHAR(15),
    email VARCHAR(150),
    address VARCHAR(255)
);
```

A. Fetch All Offices for Registration & Lookups (Flask Handler: `get_lookup_data`)

```
SELECT office_id, office_name, phone, email, address
FROM Agriculture_Offices
ORDER BY office_name ASC;
```

Query Breakdown: Executed during officer onboarding and jurisdictional filtering. Returns the complete roster of Krishi Bhavans sorted alphabetically for frontend dropdown selectors.

3. Panchayaths (Gram Panchayaths)

GET /API/PANCHAYATHS

Purpose: Models the local self-government territories (Gram Panchayaths). Each Panchayath is administratively linked to an Agriculture Office via a foreign key.

Schema Definition:

```
CREATE TABLE Panchayaths (
    panchayath_id INT AUTO_INCREMENT PRIMARY KEY,
    panchayath_name VARCHAR(150) NOT NULL,
    district VARCHAR(100) NOT NULL,
    office_id INT NOT NULL,
    FOREIGN KEY (office_id) REFERENCES Agriculture_Offices(office_id) ON DELETE RESTRICT
);
```

A. Fetch Panchayaths with Associated Office Details (Flask Handler: `get_lookup_data`)

```
SELECT panchayath_id, panchayath_name, district, office_id
FROM Panchayaths
ORDER BY panchayath_name ASC;
```

Query Breakdown: Populates dynamic selection lists on farmer registration. The `ON DELETE RESTRICT` foreign key constraint prevents accidental deletion of an office if Panchayaths depend on it.

4. Officers (Agricultural Officers Directory)

POST /API/USERS/COMPLETE-PROFILE

Purpose: Extension table for `Users` with role `OFFICER`. Maps officers to their assigned Krishi Bhavan and optional Gram Panchayath jurisdiction.

Schema Definition:

```
CREATE TABLE Officers (
    officer_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL UNIQUE,
    office_id INT NOT NULL,
    panchayath_id INT NULL,
    designation VARCHAR(100) DEFAULT 'Agricultural Officer',
    FOREIGN KEY (user_id) REFERENCES Users(user_id) ON DELETE CASCADE,
    FOREIGN KEY (office_id) REFERENCES Agriculture_Offices(office_id) ON DELETE RESTRICT,
    FOREIGN KEY (panchayath_id) REFERENCES Panchayaths(panchayath_id) ON DELETE SET NULL
);
```

A. Complete Officer Profile (Flask Handler: `handle_complete_profile`)

```
INSERT INTO Officers (user_id, office_id, panchayath_id, designation)
VALUES (%s, %s, %s, %s);
```

Query Breakdown: Links the newly registered user account to a designated office and jurisdiction. `ON DELETE CASCADE` ensures that removing a user automatically purges their officer record.

B. Resolve Officer Identity & Assigned Jurisdiction

```
SELECT o.officer_id, o.office_id, o.panchayath_id, o.designation,
       ao.office_name, p.panchayath_name
FROM Officers o
JOIN Agriculture_Offices ao ON ao.office_id = o.office_id
LEFT JOIN Panchayaths p ON p.panchayath_id = o.panchayath_id
WHERE o.user_id = %s;
```

Query Breakdown: Executed on every officer request to determine the officer's administrative boundary. `LEFT JOIN` allows officers assigned at the Krishi Bhavan level (supervising all panchayaths) to have a `NULL` `panchayath_id`.

5. Farmers (Registered Farm Operators)

POST /API/USERS/COMPLETE-PROFILE

Purpose: Extension table for `Users` with role `FARMER`. Associates the farmer with their home Gram Panchayath and residential address.

Schema Definition:

```
CREATE TABLE Farmers (
    farmer_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL UNIQUE,
    panchayath_id INT NOT NULL,
    address VARCHAR(255),
    registration_date DATE DEFAULT (CURRENT_DATE),
    FOREIGN KEY (user_id) REFERENCES Users(user_id) ON DELETE CASCADE,
    FOREIGN KEY (panchayath_id) REFERENCES Panchayaths(panchayath_id) ON DELETE RESTRICT
);
```

A. Create Farmer Profile Record

```
INSERT INTO Farmers (user_id, panchayath_id, address, registration_date)
VALUES (%s, %s, %s, CURRENT_DATE);
```

Query Breakdown: Inserts the core agricultural operator profile. `CURRENT_DATE` automatically establishes the official registration timestamp.

B. Fetch Farmer Details with Jurisdictional Hierarchy

```
SELECT f.farmer_id, f.panchayath_id, f.address, f.registration_date,
       p.panchayath_name, p.district, ao.office_id, ao.office_name
FROM Farmers f
JOIN Panchayaths p ON p.panchayath_id = f.panchayath_id
JOIN Agriculture_Offices ao ON ao.office_id = p.office_id
WHERE f.user_id = %s;
```

Query Breakdown: Multi-table `JOIN` traversing `Farmers → Panchayaths → Agriculture_Offices`. Resolves the farmer's local Gram Panchayath and supervising Krishi Bhavan in a single round-trip.

6. Farms (Land Holdings & Plots)

POST /API/FARMER/FARMS

Purpose: Stores individual agricultural plots owned or operated by a farmer, including acreage, location, soil characteristics, and irrigation infrastructure.

Schema Definition:

```
CREATE TABLE Farms (
    farm_id INT AUTO_INCREMENT PRIMARY KEY,
    farmer_id INT NOT NULL,
    farm_name VARCHAR(100),
    location VARCHAR(255) NOT NULL,
    area_acres DECIMAL(10,2) NOT NULL,
    soil_type VARCHAR(50),
    irrigation_type VARCHAR(50),
    FOREIGN KEY (farmer_id) REFERENCES Farmers(farmer_id) ON DELETE CASCADE,
    CHECK (area_acres > 0)
);
```

A. Insert Farm Plot (Flask Handler: `handle_add_farm`)

```
INSERT INTO Farms (farmer_id, farm_name, location, area_acres, soil_type, irrigation_type)
VALUES (%s, %s, %s, %s, %s, %s);
```

Query Breakdown: Adds a new parcel of agricultural land. The database enforces `CHECK (area_acres > 0)` to mathematically disallow zero or negative land holdings.

B. Update Farm Details (Flask Handler: `handle_edit_farm`)

```
UPDATE Farms
SET farm_name = %s, location = %s, area_acres = %s, soil_type = %s, irrigation_type = %s
WHERE farm_id = %s AND farmer_id = %s;
```

Query Breakdown: Modifies plot specifications. The compounding condition `WHERE farm_id = %s AND farmer_id = %s` enforces tenant isolation, preventing Farmer A from editing Farmer B's plot.

C. Retrieve Farmer's Land Parcels

```
SELECT farm_id, farm_name, location, area_acres, soil_type, irrigation_type
FROM Farms
WHERE farmer_id = %s
ORDER BY farm_name, farm_id;
```

Query Breakdown: Populates the farmer's asset dashboard and farm selection dropdowns across crop planting and expense tracking modals.

7. Crops (Master Crop Catalog)

GET /API/REFERENCE-DATA

Purpose: Canonical reference catalog defining standardized crop types, typical cultivation duration, agricultural seasons, and benchmark yields.

Schema Definition:

```
CREATE TABLE Crops (
    crop_id INT AUTO_INCREMENT PRIMARY KEY,
    crop_name VARCHAR(100) NOT NULL UNIQUE,
    crop_category VARCHAR(50),
    season VARCHAR(50),
    growth_duration_days INT,
    expected_yield_per_acre DECIMAL(10,2),
    CHECK (growth_duration_days IS NULL OR growth_duration_days > 0),
    CHECK (expected_yield_per_acre IS NULL OR expected_yield_per_acre >= 0)
);
```

A. Fetch Master Crops List (Flask Handler: `get_reference_data`)

```
SELECT crop_id, crop_name, crop_category, season, growth_duration_days, expected_yield_per_acre
FROM Crops
ORDER BY crop_name ASC;
```

Query Breakdown: Loaded by the frontend on dashboard initialization. Enables client-side auto-fill of expected harvest dates and yield calculations based on selected planting date.

8. Farm_Crops (Active & Historical Cultivation Cycles)

POST /API/FARMER/CROPS

Purpose: Represents a specific seasonal planting cycle on a farm plot. Bridges `Farms` and `Crops` and acts as the central cost-accounting center for labor, fertilizer, and income.

Schema Definition:

```
CREATE TABLE Farm_Crops (
    farm_crop_id INT AUTO_INCREMENT PRIMARY KEY,
    farm_id INT NOT NULL,
    crop_id INT NOT NULL,
    season VARCHAR(50),
    planting_date DATE,
    expected_harvest_date DATE,
    actual_harvest_date DATE,
    cultivated_area_acres DECIMAL(10,2) NOT NULL,
    status ENUM('PLANNED', 'GROWING', 'HARVESTED', 'FAILED') DEFAULT 'PLANNED',
    FOREIGN KEY (farm_id) REFERENCES Farms(farm_id) ON DELETE CASCADE,
    FOREIGN KEY (crop_id) REFERENCES Crops(crop_id) ON DELETE RESTRICT,
    CHECK (cultivated_area_acres > 0)
);
```

A. Plant a Crop Cycle (Flask Handler: `handle_plant_crop`)

```
INSERT INTO Farm_Crops (farm_id, crop_id, season, planting_date, expected_harvest_date, cultivated_a
VALUES (%s, %s, %s, %s, %s, %s, %s);
```

Query Breakdown: Initiates a crop cycle in `GROWING` or `PLANNED` state. `cultivated_area_acres` is validated to be positive and physically less than or equal to total farm acreage.

B. Edit / Update Crop Cycle Status (Flask Handler: `handle_edit_crop`)

```
UPDATE Farm_Crops
SET farm_id = %s, crop_id = %s, season = %s, planting_date = %s,
    expected_harvest_date = %s, actual_harvest_date = %s,
    cultivated_area_acres = %s, status = %s
WHERE farm_crop_id = %s;
```

Query Breakdown: Allows status progression from `GROWING → HARVESTED` or `FAILED`, recording actual harvest dates for agronomic performance analysis.

C. Comprehensive Crop Overview with Multi-Table Joins

```
SELECT fc.farm_crop_id, fc.farm_id, f.farm_name, fc.crop_id, c.crop_name,
       c.crop_category, fc.season, fc.planting_date, fc.expected_harvest_date,
       fc.actual_harvest_date, fc.cultivated_area_acres, fc.status
FROM Farm_Crops AS fc
JOIN Farms AS f ON f.farm_id = fc.farm_id
JOIN Crops AS c ON c.crop_id = fc.crop_id
WHERE f.farmer_id = %s
ORDER BY fc.planting_date DESC, fc.farm_crop_id DESC;
```

Query Breakdown: Joins `Farm_Crops` with parent `Farms` and master `Crops`. Filters strictly by `f.farmer_id` to provide the farmer's complete agricultural activity log.

9. Labour_Types (Agricultural Work Classifications)

GET /API/REFERENCE-DATA

Purpose: Master reference table categorizing manual and mechanized agricultural tasks (e.g., Tilling, Sowing, Weeding, Harvesting) and standard daily wage rates.

Schema Definition:

```
CREATE TABLE Labour_Types (
    labour_id INT AUTO_INCREMENT PRIMARY KEY,
    labour_name VARCHAR(100) NOT NULL UNIQUE,
    standard_daily_rate DECIMAL(10,2)
);
```

A. Fetch Standard Labor Rates

```
SELECT labour_id, labour_name, standard_daily_rate
FROM Labour_Types
ORDER BY labour_name ASC;
```

Query Breakdown: Feeds frontend wage calculators so farmers can estimate labor outlays based on work days and worker headcount.

10. Fertilizers (Nutrient & Soil Amendments)

GET /API/REFERENCE-DATA

Purpose: Master reference catalog of organic manures and chemical fertilizers (e.g., Urea, DAP, Potash, Compost) and standard measurement units.

Schema Definition:

```
CREATE TABLE Fertilizers (
    fertilizer_id INT AUTO_INCREMENT PRIMARY KEY,
    fertilizer_name VARCHAR(120) NOT NULL UNIQUE,
    fertilizer_type VARCHAR(50),
    unit VARCHAR(20) DEFAULT 'kg'
);
```

A. Fetch Master Fertilizers List

```
SELECT fertilizer_id, fertilizer_name, fertilizer_type, unit
FROM Fertilizers
ORDER BY fertilizer_name ASC;
```

Query Breakdown: Populates fertilizer selection pickers on the farmer dashboard with pre-configured units (kg, liters, bags).

11. Rental_Types (Machinery & Tool Rentals)

GET /API/REFERENCE-DATA

Purpose: Standardized classifications for hired farm equipment (e.g., Tractor, Harvester, Power Tiller, Drip Irrigation Kits).

Schema Definition:

```
CREATE TABLE Rental_Types (
    rental_id INT AUTO_INCREMENT PRIMARY KEY,
    rental_name VARCHAR(120) NOT NULL UNIQUE,
    description VARCHAR(255)
);
```

A. Fetch Machinery Rental Categories

```
SELECT rental_id, rental_name, description
FROM Rental_Types
ORDER BY rental_name ASC;
```

Query Breakdown: Categorizes rental equipment expenses under the `Expenses` entity.

12. Energy_Types (Power & Fuel Utilities)

GET /API/REFERENCE-DATA

Purpose: Standardized classifications for agricultural power sources (e.g., Grid Electricity, Diesel Fuel, Solar Pump Power).

Schema Definition:

```
CREATE TABLE Energy_Types (
    energy_id INT AUTO_INCREMENT PRIMARY KEY,
    energy_name VARCHAR(100) NOT NULL UNIQUE
);
```

A. Fetch Energy Source Categories

```
SELECT energy_id, energy_name
FROM Energy_Types
ORDER BY energy_name ASC;
```

Query Breakdown: Categorizes power and fuel expenditures under the `Expenses` entity.

13. Expenses (Operational Expenditures)

POST /API/FARMER/EXPENSES

Purpose: Granular ledger recording general farm expenses across Seeds, Rental, Energy, Irrigation, Transport, and Miscellaneous outlays.

Schema Definition:

```
CREATE TABLE Expenses (
    expense_id INT AUTO_INCREMENT PRIMARY KEY,
    farm_id INT NOT NULL,
    farm_crop_id INT NULL,
    expense_category ENUM('SEEDS', 'RENTAL', 'ENERGY', 'IRRIGATION', 'TRANSPORT', 'OTHER') NOT NULL,
    rental_id INT NULL,
    energy_id INT NULL,
    amount DECIMAL(12,2) NOT NULL,
    expense_date DATE NOT NULL,
    description VARCHAR(255),
    FOREIGN KEY (farm_id) REFERENCES Farms(farm_id) ON DELETE CASCADE,
    FOREIGN KEY (farm_crop_id) REFERENCES Farm_Crops(farm_crop_id) ON DELETE SET NULL,
    FOREIGN KEY (rental_id) REFERENCES Rental_Types(rental_id) ON DELETE SET NULL,
    FOREIGN KEY (energy_id) REFERENCES Energy_Types(energy_id) ON DELETE SET NULL
);
```

A. Record Operational Expense (Flask Handler: `handle_add_expense`)

```
INSERT INTO Expenses (farm_id, farm_crop_id, expense_category, rental_id, energy_id, amount, expense
VALUES (%s, %s, %s, %s, %s, %s, %s, %s);
```

Query Breakdown: Adds an expense entry linked optionally to a specific crop cycle (`farm_crop_id`) or directly to the overarching farm (`farm_id`). `ON DELETE SET NULL` ensures financial audit logs are preserved even if a rental type is decommissioned.

B. Detailed Expense Report with Left Outer Joins

```
SELECT e.expense_id, e.farm_id, f.farm_name, e.farm_crop_id, c.crop_name,
       e.expense_category, e.rental_id, rt.rental_name, e.energy_id, et.energy_name,
       e.amount, e.expense_date, e.description
FROM Expenses AS e
JOIN Farms AS f ON f.farm_id = e.farm_id
LEFT JOIN Farm_Crops AS fc ON fc.farm_crop_id = e.farm_crop_id
LEFT JOIN Crops AS c ON c.crop_id = fc.crop_id
LEFT JOIN Rental_Types AS rt ON rt.rental_id = e.rental_id
LEFT JOIN Energy_Types AS et ON et.energy_id = e.energy_id
WHERE f.farmer_id = %s
ORDER BY e.expense_date DESC, e.expense_id DESC;
```

Query Breakdown: Utilizes four `LEFT JOIN` clauses so that general expenses without a linked crop cycle, rental equipment, or energy source still appear accurately with `NULL` values cleanly converted to empty strings in JSON.

14. Farm_Crop_Labour (Labor Work Logs)

POST /API/FARMER/LABOUR

Purpose: Tracks field worker deployment, days worked, and financial compensation for specific crop cultivation cycles.

Schema Definition:

```
CREATE TABLE Farm_Crop_Labour (
    labour_usage_id INT AUTO_INCREMENT PRIMARY KEY,
    farm_crop_id INT NOT NULL,
    labour_id INT NOT NULL,
    work_date DATE NOT NULL,
    worker_count INT NOT NULL DEFAULT 1,
    days_worked DECIMAL(6,2) NOT NULL DEFAULT 1,
    total_cost DECIMAL(12,2) NOT NULL,
    FOREIGN KEY (farm_crop_id) REFERENCES Farm_Crops(farm_crop_id) ON DELETE CASCADE,
    FOREIGN KEY (labour_id) REFERENCES Labour_Types(labour_id) ON DELETE RESTRICT
);
```

A. Insert Labor Deployment Log (Flask Handler: `handle_add_labour`)

```
INSERT INTO Farm_Crop_Labour (farm_crop_id, labour_id, work_date, worker_count, days_worked, total_c
VALUES (%s, %s, %s, %s, %s, %s);
```

Query Breakdown: Persists wage costs. Flask validates that `total_cost = worker_count * days_worked * daily_rate` or records actual agreed payments.

B. Fetch Labor Cost Breakdown for Farmer

```
SELECT fcl.labour_usage_id, fcl.farm_crop_id, f.farm_name, c.crop_name,
       fcl.labour_id, lt.labour_name, fcl.work_date, fcl.worker_count,
       fcl.days_worked, fcl.total_cost
FROM Farm_Crop_Labour AS fcl
JOIN Farm_Crops AS fc ON fc.farm_crop_id = fcl.farm_crop_id
JOIN Farms AS f ON f.farm_id = fc.farm_id
JOIN Crops AS c ON c.crop_id = fc.crop_id
JOIN Labour_Types AS lt ON lt.labour_id = fcl.labour_id
WHERE f.farmer_id = %s
ORDER BY fcl.work_date DESC, fcl.labour_usage_id DESC;
```

Query Breakdown: Multi-table join displaying labor utilization per crop cycle, feeding directly into total farm operational expenditure calculations.

15. Farm_Crop_Fertilizer (Nutrient Application Logs)

POST /API/FARMER/FERTILIZERS

Purpose: Records physical application of organic and chemical fertilizers to crops, tracking dosages and associated costs.

Schema Definition:

```
CREATE TABLE Farm_Crop_Fertilizer (
    application_id INT AUTO_INCREMENT PRIMARY KEY,
    farm_crop_id INT NOT NULL,
    fertilizer_id INT NOT NULL,
    application_date DATE NOT NULL,
    quantity DECIMAL(12,2) NOT NULL,
    total_cost DECIMAL(12,2) NOT NULL DEFAULT 0,
    FOREIGN KEY (farm_crop_id) REFERENCES Farm_Crops(farm_crop_id) ON DELETE CASCADE,
    FOREIGN KEY (fertilizer_id) REFERENCES Fertilizers(fertilizer_id) ON DELETE RESTRICT
);
```

A. Log Fertilizer Application (Flask Handler: `handle_add_fertilizer`)

```
INSERT INTO Farm_Crop_Fertilizer (farm_crop_id, fertilizer_id, application_date, quantity, total_cos
VALUES (%s, %s, %s, %s, %s);
```

Query Breakdown: Captures input dosages and expenditure. `ON DELETE RESTRICT` on `fertilizer_id` preserves audit records even if catalog definitions are modified.

B. Retrieve Fertilizer Application History

```
SELECT fcf.application_id, fcf.farm_crop_id, f.farm_name, c.crop_name,
       fcf.fertilizer_id, fert.fertilizer_name, fert.fertilizer_type, fert.unit,
       fcf.application_date, fcf.quantity, fcf.total_cost
FROM Farm_Crop_Fertilizer AS fcf
JOIN Farm_Crops AS fc ON fc.farm_crop_id = fcf.farm_crop_id
JOIN Farms AS f ON f.farm_id = fc.farm_id
JOIN Crops AS c ON c.crop_id = fc.crop_id
JOIN Fertilizers AS fert ON fert.fertilizer_id = fcf.fertilizer_id
WHERE f.farmer_id = %s
ORDER BY fcf.application_date DESC, fcf.application_id DESC;
```

Query Breakdown: Resolves fertilizer specifications (commercial name, chemical/organic type, unit) alongside the crop name and farm plot.

16. Income (Harvest Sales & Revenues)

POST /API/FARMER/INCOME

Purpose: Financial revenue ledger tracking crop yields sold to commercial buyers or government mandis. Features an automated stored generated column for total income.

Schema Definition:

```
CREATE TABLE Income (
    income_id INT AUTO_INCREMENT PRIMARY KEY,
    farm_crop_id INT NOT NULL,
    sale_date DATE NOT NULL,
    quantity_sold DECIMAL(12,2) NOT NULL,
    unit VARCHAR(20) DEFAULT 'kg',
    price_per_unit DECIMAL(12,2) NOT NULL,
    total_income DECIMAL(14,2) GENERATED ALWAYS AS (quantity_sold * price_per_unit) STORED,
    buyer_name VARCHAR(150),
    FOREIGN KEY (farm_crop_id) REFERENCES Farm_Crops(farm_crop_id) ON DELETE CASCADE
);
```

A. Record Harvest Sale (Flask Handler: `handle_add_income`)

```
INSERT INTO Income (farm_crop_id, sale_date, quantity_sold, unit, price_per_unit, buyer_name)
VALUES (%s, %s, %s, %s, %s, %s);
```

Query Breakdown: Notice that `total_income` is **omitted** from the `INSERT` statement! MySQL automatically evaluates `quantity_sold * price_per_unit` and writes the calculated value directly to the storage block as a `STORED` column. This eliminates floating point rounding discrepancies between Python and MySQL.

B. Fetch Revenue Ledger for Farmer

```
SELECT inc.income_id, inc.farm_crop_id, f.farm_name, c.crop_name,
       inc.sale_date, inc.quantity_sold, inc.unit, inc.price_per_unit,
       inc.total_income, inc.buyer_name
FROM Income AS inc
JOIN Farm_Crops AS fc ON fc.farm_crop_id = inc.farm_crop_id
JOIN Farms AS f ON f.farm_id = fc.farm_id
JOIN Crops AS c ON c.crop_id = fc.crop_id
WHERE f.farmer_id = %s
ORDER BY inc.sale_date DESC, inc.income_id DESC;
```

Query Breakdown: Retrieves sales entries with linked crop varieties and farms, powering the farmer's gross revenue metrics.

17. Subsidies (Government Schemes Catalog)

GET /API/REFERENCE-DATA

Purpose: Master catalog of state and central agricultural subsidy schemes (e.g., Organic Fertilizer Grant, Subsidized Seeds, Solar Pump Subvention) with eligibility terms.

Schema Definition:

```
CREATE TABLE Subsidies (
    subsidy_id INT AUTO_INCREMENT PRIMARY KEY,
    subsidy_name VARCHAR(150) NOT NULL,
    description TEXT,
    amount DECIMAL(12,2) NOT NULL,
    start_date DATE,
    end_date DATE,
    eligibility_criteria TEXT
);
```

A. Fetch Active Subsidy Schemes

```
SELECT subsidy_id, subsidy_name, description, amount, start_date, end_date, eligibility_criteria
FROM Subsidies
WHERE (CURRENT_DATE BETWEEN start_date AND end_date) OR start_date IS NULL
ORDER BY subsidy_name ASC;
```

Query Breakdown: Filters schemes dynamically using MySQL's `CURRENT_DATE` so that farmers only view currently active financial aid programs.

18. Farmer_Subsidies (Applications & Officer Review)

`POST /API/FARMER/SUBSIDIES` `POST /API/OFFICER/SUBSIDIES/REVIEW`

Purpose: Bridges farmers and government subsidies. Manages the full application lifecycle: `PENDING → APPROVED / REJECTED → PAID`, officer jurisdiction validation, and disbursement records.

Schema Definition:

```
CREATE TABLE Farmer_Subsidies (
    farmer_subsidy_id INT AUTO_INCREMENT PRIMARY KEY,
    farmer_id INT NOT NULL,
    subsidy_id INT NOT NULL,
    officer_id INT NULL,
    application_date DATE NOT NULL,
    status ENUM('PENDING', 'APPROVED', 'REJECTED', 'PAID') DEFAULT 'PENDING',
    approved_amount DECIMAL(12,2),
    payment_date DATE,
    remarks VARCHAR(255),
    FOREIGN KEY (farmer_id) REFERENCES Farmers(farmer_id) ON DELETE CASCADE,
    FOREIGN KEY (subsidy_id) REFERENCES Subsidies(subsidy_id) ON DELETE RESTRICT,
    FOREIGN KEY (officer_id) REFERENCES Officers(officer_id) ON DELETE SET NULL,
    UNIQUE (farmer_id, subsidy_id)
);
```

A. Apply for Subsidy (Flask Handler: `handle_apply_subsidy`)

```
INSERT INTO Farmer_Subsidies (farmer_id, subsidy_id, application_date, status)
VALUES (%s, %s, CURRENT_DATE, 'PENDING');
```

Query Breakdown: Submits an application in `PENDING` state. The table constraint `UNIQUE (farmer_id, subsidy_id)` prevents accidental or fraudulent duplicate submissions for the same scheme.

B. Officer Review & Approval (Flask Handler: `handle_review_subsidy`)

```
UPDATE Farmer_Subsidies AS fs
SET fs.status = %s,
    fs.approved_amount = %s,
    fs.remarks = %s,
    fs.officer_id = %s,
    fs.payment_date = %s
WHERE fs.farmer_subsidy_id = %s;
```

Query Breakdown: Updates review status. Before executing this query, Flask performs a strict jurisdictional check ensuring the applicant farmer resides within the officer's designated Krishi Bhavan / Gram Panchayath.

C. Farmer View of Applied Subsidies with Reviewing Officer Attribution

```
SELECT fs.farmer_subsidy_id, s.subsidy_id, s.subsidy_name, s.description,
       s.amount, fs.status, fs.application_date, fs.approved_amount,
       fs.payment_date, fs.remarks, off_u.full_name AS officer_name,
       p.panchayath_name, o.designation AS officer_designation
FROM Farmer_Subsidies AS fs
JOIN Subsidies AS s ON s.subsidy_id = fs.subsidy_id
JOIN Farmers AS f ON f.farmer_id = fs.farmer_id
JOIN Panchayaths AS p ON p.panchayath_id = f.panchayath_id
LEFT JOIN Officers AS o ON o.officer_id = fs.officer_id
LEFT JOIN Users AS off_u ON off_u.user_id = o.user_id
WHERE fs.farmer_id = %s
ORDER BY fs.application_date DESC, fs.farmer_subsidy_id DESC;
```

Query Breakdown: Links the subsidy application to the reviewing agricultural officer's name and designation using `LEFT JOIN` so that pending applications (which do not yet have an assigned officer) display properly.

19. Officer_Farmer_Directory (Denormalized Architectural View)

DATABASE VIEW

Purpose: A database view providing pre-joined, high-performance read access across administrative hierarchies (Krishi Bhavan → Gram Panchayath → Farmer → Officer).

View DDL Definition:

```
CREATE VIEW Officer_Farmer_Directory AS
SELECT
    off.office_id,
    off.office_name,
    o.officer_id,
    u_off.user_id AS officer_user_id,
    u_off.full_name AS officer_name,
    p.panchayath_id,
    p.panchayath_name,
    p.district,
    f.farmer_id,
    u_farm.full_name AS farmer_name,
    u_farm.email AS farmer_email,
    u_farm.phone AS farmer_phone,
    f.address AS farmer_address,
    f.registration_date
FROM Officers o
JOIN Users u_off ON o.user_id = u_off.user_id
JOIN Agriculture_Offices off ON o.office_id = off.office_id
JOIN Panchayaths p ON off.office_id = p.office_id
JOIN Farmers f ON p.panchayath_id = f.panchayath_id
JOIN Users u_farm ON f.user_id = u_farm.user_id;
```

Query Breakdown: Encapsulates 5 distinct relational table joins into a single querying target. Allows officers to query all farmers under their jurisdiction with simple `WHERE office_id = %s` predicates without repeatedly writing complex multi-table joins in application code.

5. Advanced Cross-Table Aggregation Queries

Beyond standard CRUD operations, the Agriculture Management System executes sophisticated analytical and financial queries that perform multi-table calculations and rollups in real-time.

5.1 Financial Net Balance Aggregation

To deliver the farmer's financial dashboard in `get_farmer_overview`, Flask compiles an exact financial balance across 5 independent entity streams:

```
# Aggregation logic in backend/app.py
total_expenses    = sum(float(e["amount"] or 0) for e in expenses)
total_fert_cost   = sum(float(f["total_cost"] or 0) for f in fertilizers)
total_labour_cost = sum(float(l["total_cost"] or 0) for l in labour)
total_income      = sum(float(i["total_income"] or 0) for i in income)
total_subsidies   = sum(float(s["approved_amount"] or 0) for s in subsidies if s["status"] in ("APPROVED"

overall_cost = total_expenses + total_fert_cost + total_labour_cost
net_balance  = (total_income + total_subsidies) - overall_cost
```

This multi-table computation provides the farmer with an accurate, real-time Profit/Loss statement: $$\text{Net Balance} = (\text{Crop Sales} + \text{Approved Subsidies}) - (\text{General Expenses} + \text{Fertilizer Outlays} + \text{Labor Wages})$$

5.2 Officer Farmer Overview with Correlated Subqueries & String Concatenation

In `get_officer_farmers`, Flask executes an advanced analytical query that compiles total farms count, acreage sum, and distinct crop varieties per farmer using `GROUP_CONCAT`:

```
SELECT f.farmer_id,
       u.full_name AS farmer_name,
       u.email AS farmer_email,
       u.phone AS farmer_phone,
       p.panchayath_id,
       p.panchayath_name,
       ao.office_id,
       ao.office_name,
       f.address AS farmer_address,
       f.registration_date,
       COALESCE(farm_stats.farms_count, 0) AS farms_count,
       COALESCE(farm_stats.total_acres, 0) AS total_acres,
       COALESCE(farm_stats.farms_summary, '') AS farms_summary,
       COALESCE(crop_stats.crops_count, 0) AS crops_count,
       COALESCE(crop_stats.crops_summary, '') AS crops_summary
FROM Farmers f
JOIN Users u ON u.user_id = f.user_id
JOIN Panchayaths p ON p.panchayath_id = f.panchayath_id
JOIN Agriculture_Offices ao ON ao.office_id = p.office_id
LEFT JOIN (
    SELECT farmer_id,
           COUNT(farm_id) AS farms_count,
           SUM(area_acres) AS total_acres,
           GROUP_CONCAT(farm_name SEPARATOR ', ') AS farms_summary
    FROM Farms
    GROUP BY farmer_id
) farm_stats ON farm_stats.farmer_id = f.farmer_id
LEFT JOIN (
    SELECT f2.farmer_id,
           COUNT(DISTINCT fc.farm_crop_id) AS crops_count,
           GROUP_CONCAT(DISTINCT c.crop_name SEPARATOR ', ') AS crops_summary
    FROM Farm_Crops fc
    JOIN Farms f2 ON f2.farm_id = fc.farm_id
    JOIN Crops c ON c.crop_id = fc.crop_id
    GROUP BY f2.farmer_id
) crop_stats ON crop_stats.farmer_id = f.farmer_id
WHERE p.office_id = %s
ORDER BY p.panchayath_name, u.full_name;
```

Deep SQL Query Mechanics

- **LEFT JOIN with Pre-Aggregated Subqueries:** Instead of joining `Farms` directly (which would multiply farmer rows by farm count and crop count, creating an exponential Cartesian product), the query pre-groups farms and crops in subqueries. This preserves strict 1-row-per-farmer cardinality.
- **COALESCE Function:** Guarantees that farmers with zero farms or crops return `0` or empty strings instead of `NULL`, ensuring the frontend JSON parser receives clean numeric types.
- **GROUP_CONCAT Function:** Concatenates distinct farm plot names (e.g., "*North Acre, Riverside Plot*") into a single string for quick badge display in the officer's directory table.

6. Viva Voce & Technical Defense Q&A

Use these questions and answers to defend the system architecture, Flask integration, and database design before evaluators.

Q1: What is the exact role of Flask in your project?
Answer: Flask serves as the Application Tier (Tier 2). It routes incoming HTTP requests from the browser, parses JSON payloads from `request.environ`, enforces authentication and Role-Based Access Control via JWT, manages transactional database sockets, executes parameterized SQL queries, and packages query results into standardized JSON responses with appropriate HTTP status codes.

Q2: How does Flask connect to MySQL? Do you use an ORM like SQLAlchemy?
Answer: We chose raw parameterized SQL queries using official `mysql.connector` over an ORM. This allows explicit control over query performance, guarantees complete visibility into all SQL execution plans, avoids ORM "N+1 query" pitfalls, and enables sophisticated DBMS features like stored generated columns, explicit transactions, and database views.

Q3: How does your database ensure that a farmer cannot delete a crop if financial records exist?
Answer: Through foreign key referential integrity constraints. For example, `Farm_Crops` is referenced by `Income`, `Farm_Crop_Labour`, and `Farm_Crop_Fertilizer`. When deleting a farm, dependent crop cycles and their financial records cascade predictably (`ON DELETE CASCADE`) or prevent orphaned records, maintaining absolute relational consistency.

Q4: How are passwords secured in the database?
Answer: Passwords are never stored in plain text. We utilize PBKDF2 with HMAC-SHA256, 310,000 iterations, and a 16-byte random salt generated via Python's `secrets` module. On login, the hash is compared using `hmac.compare_digest` to prevent timing attacks.

Q5: How does Flask handle concurrent requests from multiple farmers?
Answer: In local development, Flask's Werkzeug server runs with multi-threading enabled by default (`threaded=True`), spawning thread workers for concurrent browser requests. In cloud production (Railway/Docker), Gunicorn runs 3 parallel pre-forked worker processes, allowing the system to handle concurrent web and API traffic smoothly without socket contention.
